EE6221 Robotics and Intelligent Sensors

Beginner Notes: Manipulator Kinematics

Living course note

Updated: 24 August 2026

Scope and sources. This self-contained note develops the supplied manipulator material from first vocabulary through direct and inverse kinematics, elementary trajectory planning, Jacobians, and singularities. It uses `resources/lecture-01-manipulator-kinematics.pdf`, pp. 1–156, `resources/course-outline-part-01.pdf`, and the Week 1/2 transcripts as supporting teaching context. Newly archived public slides, assignments, and examinations are recorded in `STATUS.md` but are not treated as covered here until independently reviewed.

Contents

1 What is a robot manipulator?

For this course, a robot is an industrial, reprogrammable, multifunctional manipulator. A manipulator is a chain of rigid *links* joined by actuated *joints*; the final link carries an *end effector*, such as a gripper, hand, or tool. Moving the joints changes the end effector's *pose*: its position and orientation.

The two fundamental joint types are:

- **Revolute (R):** the link rotates about an axis; its joint coordinate is an angle.
- **Prismatic (P):** the link slides along an axis; its joint coordinate is a displacement.

The number of independently controlled joint coordinates is the robot's degrees of freedom (DoF). The first few joints often position the wrist, while wrist joints orient the tool. A robot can be useful with fewer than six DoF, but then it may be unable to realise arbitrary position-and-orientation requests.

2 Why and how robots are used

Robots are particularly useful for repetitive, hazardous, precise, or reconfigurable tasks. In contrast to a hard-automation machine designed for one fixed task, a reprogrammable robot can change task by changing its program and tooling. This flexibility has a cost: it may be less economical than dedicated equipment at very high production volumes, while manual work can be preferable at very low volumes.

Common manipulator structures are named from the major joint types: Cartesian PPP, cylindrical RPP, spherical RRP, SCARA RRP, and articulated RRR. Their structure affects the workspace: the set of tool poses that can be reached. The workspace is not simply a ball around the base. Joint limits, link dimensions, collisions, and orientation requirements restrict it.

Useful specifications include payload (allowable load), reach, stroke, speed, repeatability, accuracy, and resolution. *Accuracy* is closeness to the commanded pose; *repeatability* is how consistently the robot returns to a pose. A robot may be repeatable but inaccurate if a systematic calibration error shifts every placement.

Source: `resources/lecture-01-manipulator-kinematics.pdf`, pp. 1–22; `resources/week-01-transcript.txt`..

3 Why coordinate frames are essential

Saying "the tool is at $(1, 2, 3)$" is incomplete: those numbers only have meaning relative to a coordinate frame. We write $\{A\}$ for a base or reference frame and $\{B\}$ for a frame attached to a robot link. A physical vector is the same geometric object, but its components change when expressed in another frame.

If ${}^{A}R_{B}$ describes frame $\{B\}$ relative to frame $\{A\}$, then

$$^{A}v = {}^{A}R_{B}\,{}^{B}v.$$

The columns of ${}^{A}R_{B}$ are the x_B, y_B, and z_B axes written in frame $\{A\}$. This gives an immediate interpretation: a rotation matrix tells an observer in $\{A\}$ which way the axes attached to $\{B\}$ point.

4 Rotations and homogeneous transformations

A valid three-dimensional rotation matrix satisfies

$$R^\top R = I, \qquad R^{-1} = R^\top, \qquad \det R = +1.$$

For example, a right-handed rotation about the z axis is

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Matrix order matters. Rotations about fixed axes premultiply an existing orientation; rotations about moving (body) axes postmultiply. Always state the frame and physical order instead of relying on a memorised product.

Rotation alone cannot account for different origins. If the origin of $\{B\}$ is at ${}^{A}p_{B}$, a point P satisfies

$$^{A}p_{P} = {}^{A}R_{B}\,{}^{B}p_{P} + {}^{A}p_{B}.$$

Homogeneous coordinates combine both operations:

$$^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}p_{B} \\ 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} {}^{A}p_{P} \\ 1 \end{bmatrix} = {}^{A}T_{B} \begin{bmatrix} {}^{B}p_{P} \\ 1 \end{bmatrix}.$$

The inverse is

$$({}^{A}T_{B})^{-1} = \begin{bmatrix} ({}^{A}R_{B})^{\mathsf{T}} & -({}^{A}R_{B})^{\mathsf{T}}\,{}^{A}p_{B} \\ 0 & 1 \end{bmatrix}.$$

The translated term must be rotated when inverting; simply negating it is a common mistake.

Source: `resources/lecture-01-manipulator-kinematics.pdf`, pp. 23–52; `resources/week-01-transcript.txt`..

5 Direct kinematics and D–H construction

Direct kinematics asks: if the joint coordinates $q_1, \ldots, q_n$ are known, where is the tool? The answer is the product

$${}^{0}T_{n}(q) = {}^{0}T_{1}(q_1)\,{}^{1}T_{2}(q_2)\cdots{}^{n-1}T_{n}(q_n).$$

The superscripts and subscripts behave like frame labels: adjacent labels cancel in a valid chain. If a product does not follow a connected route from the source frame to the destination frame, its physical meaning is suspect.

The standard Denavit–Hartenberg construction represents each adjacent-link transform by four operations:

$${}^{i-1}T_{i} = R_z(\theta_i)\,T_z(d_i)\,T_x(a_i)\,R_x(\alpha_i).$$

Here θ_i rotates about z_{i-1}, d_i translates along z_{i-1}, a_i translates along x_i, and α_i rotates about x_i. For a revolute joint, θ_i is the variable; for a prismatic joint, d_i is the variable. The other quantities describe fixed link geometry.

Use this construction order:

1. Draw every joint axis and assign it as a z axis.
2. Choose each x_i along the common normal from z_{i-1} to z_i; intersecting axes give $a_i = 0$.
3. Complete a right-handed frame with $y_i = z_i \times x_i$.
4. Fill the D–H table before multiplying matrices.
5. Check the final rotation block is orthonormal and the translation has length units.

For a planar two-revolute-link arm,

$$x = a_1 \cos q_1 + a_2 \cos(q_1 + q_2), \qquad y = a_1 \sin q_1 + a_2 \sin(q_1 + q_2).$$

The second link uses the accumulated angle $q_1 + q_2$ because its orientation is measured after both joint rotations.

Source: `resources/lecture-01-manipulator-kinematics.pdf`, pp. 53–86..

6 Inverse kinematics: pose to joint coordinates

Inverse kinematics reverses the question: given a desired tool transform ${}^{0}T_{n}^{*}$, find q such that ${}^{0}T_{n}(q) = {}^{0}T_{n}^{*}$. This is not ordinary matrix inversion because q appears inside trigonometric functions and the mapping can be many-to-one.

Before solving equations, check three forms of feasibility:

- the desired position lies in the work envelope;
- the robot has enough orientation freedom for the desired tool orientation;
- each candidate respects joint limits and practical collision constraints.

An IK problem can have no solution, one solution, or multiple branches such as elbow-up and elbow-down. Use $\operatorname{atan2}(y, x)$ when reconstructing an angle: unlike $\arctan(y/x)$, it preserves quadrant information and remains meaningful when $x = 0$.

Analytic IK uses robot geometry to isolate variables and enumerate branches. Numerical IK instead iteratively reduces pose error, commonly through a Jacobian-based update. Numerical methods generalise better but need an initial guess, a convergence test, joint-limit handling, and a strategy near singularities.

Source: `resources/lecture-01-manipulator-kinematics.pdf`*, pp. 87–117..*

7 From a geometric path to a timed trajectory

A path specifies where the tool travels; a trajectory also specifies when it occupies each point. Writing $w(t) = \Gamma(s(t))$ separates the geometric curve Γ from the timing law $s(t)$. This distinction lets the same collision-free path be reused with slower or faster motion.

For one segment, a cubic polynomial

$$w(t) = at^3 + bt^2 + ct + d, qquad 0 \le t \le T,$$

can satisfy two endpoint positions and two endpoint velocities. Componentwise, for $w(0) = w_0$, $\dot{w}(0) = v_0$, $w(T) = w_1$, and $\dot{w}(T) = v_1$,

$$\begin{aligned} d &= w_0, & c &= v_0, \\ a &= \frac{2(w_0 - w_1) + T(v_0 + v_1)}{T^3}, & b &= \frac{3(w_1 - w_0) - T(2v_0 + v_1)}{T^2}. \end{aligned}$$

The powers of T are a dimensional check: a has units of position/time^3 and b position/time^2. Multi-segment motion should join with suitable velocity and acceleration continuity; matching positions alone can still create abrupt motion.

Source: `resources/lecture-01-manipulator-kinematics.pdf`*, pp. 118–132..*

8 Jacobians, velocity control, and singularities

If the task coordinate is $x = w(q)$, differentiating with respect to time gives

$$\dot{x} = J(q)\dot{q}, \qquad J(q) = \frac{\partial w}{\partial q}.$$

Column j of J is the instantaneous task-space motion produced by unit speed at joint j while all other joints are fixed. This is a local linear relationship at the current configuration, not an exact formula for a large finite displacement.

When an exact inverse does not exist, the Moore–Penrose pseudoinverse gives a least-squares or minimum-norm rate command:

$$\dot{q} = J^\dagger \dot{x}.$$

For a redundant robot, null-space motion $(I - J^\dagger J)z$ can change joint posture without changing the commanded first-order tool motion. This can help avoid limits or improve dexterity, but only if the secondary objective is chosen deliberately.

A singular configuration occurs when J loses rank. Then at least one task-space direction becomes unattainable instantaneously, and a pseudoinverse command may demand very large joint rates near the singularity. The smallest singular value of J is a useful warning: as it approaches zero, the mapping becomes ill-conditioned. In practice, slow down, re-plan, or use a damped least-squares inverse rather than trusting an unstable ordinary inverse.

Source: `resources/lecture-01-manipulator-kinematics.pdf`*, pp. 133–156..*

9 Full-course learning checklist

- Can I distinguish a link, joint, end effector, pose, and workspace?
- Can I explain why a reprogrammable robot differs from hard automation?
- Can I state the source and target frame for every rotation or transformation?
- Can I explain why the columns of a rotation matrix are axes expressed in another frame?
- Can I invert a homogeneous transform without forgetting to rotate the translation?
- Can I assign D–H frames, identify the variable parameter, and check the transform chain?
- Before solving IK, can I check reachability, orientation freedom, limits, and solution branches?
- Can I distinguish path geometry from trajectory timing and check cubic coefficient units?
- Can I explain $\dot{x} = J\dot{q}$ and what rank loss means physically?